

Wind and the loss rate

What a wind does to the chance of losing the home base on a 100×100 map, measured against the no-UAV baseline that loses it every time.

Experiment	20260810_wind
Date	10 August 2026
Commit	d08da5f851ec87026d7d973b159e29d34de88689
Simulator	Wildfire-UAVSim, mesa 1.x
Compared against	20260810_baseline – 100.0% loss over 5 000 runs
Measured	73 arms, 1 000 runs each, seeds 3 000 000 – 3 000 999
Interval	Wilson score, 95%

Summary

Wind is not a difficulty dial over most of its range. From $\mu = 0$ to $\mu = 0.75$ the loss rate is indistinguishable from the no-wind baseline in all four cardinal directions — 99.5% or above, against the baseline's 100.0% — even though by $\mu = 0.75$ the wind has discarded three quarters of every off-axis spread weight and nearly doubled the front speed.

Above $\mu = 0.85$ the loss rate collapses, and **how far it collapses depends more on which way the wind blows than on how hard.** At $\mu = 0.9$ a south or west wind still loses 97% of runs while a north or east wind loses 65%. That factor of five is a property of the map rather than of the weather: the base sits a quarter of the way into the grid, so two directions have 73 rows of ground upwind of it to draw ignitions from and two have 25.

The mechanism is not that wind keeps fire away from the base. A cell burns for one update and goes out, so the fire is a travelling wave; the base takes one point of damage per step the wave stands on it and is lost at four. Wind makes the fire faster *and* thinner, and a thin fast fire crosses the base in two or three steps and leaves. The base survives because it was singed rather than besieged.

Contents

1. Objective and result
2. What the wind does in the model
3. Parameters
4. Method
5. Results
6. Comparison with the baseline
7. Validity
8. What was not run
9. Reproducing

1. Objective and result

The no-UAV baseline established that this configuration is unwinnable: 5 000 runs, 5 000 losses, a 95% Wilson interval of 99.92 – 100.00. This experiment asks what the wind does to that, and how much of the answer is the wind rather than the map.

The question matters for two reasons. The first is practical: wind is one of the few settings that could soften a scenario which is otherwise saturated at 100%, and a saturated baseline cannot discriminate between UAV policies at the weak end. The second is that the earlier calibration experiment recorded, as a counterintuitive aside, that stronger wind made the scenario *easier*. That is worth either confirming with numbers or explaining properly, and it turns out to need both.

Result. Wind changes nothing the base can detect until $\mu > 0.75$. Above 0.85 it collapses the loss rate, by a factor that depends on direction more than on strength: at $\mu = 0.9$ the loss rate is 97.0% with a south wind and 64.6% with a north one, on the same 1 000 fires. At $\mu = 1$ a cardinal wind leaves 0.3 – 1.6% and a diagonal wind leaves 6.8 – 20.3%.

The rest of this document sets out the arithmetic that produces that shape (§2), what was held fixed and what was swept (§3, §4), the measurements (§5), how they compare with the baseline (§6), the four checks that the surface is real rather than an artefact of a setting that failed to apply (§7), and what remains unmeasured (§8).

2. What the wind does in the model

Every result below follows from about five lines of arithmetic in `sim/fire_spread.py` and one property of `sim/agents/fire.py` that is easy to miss. They are worth setting out before the measurements rather than after them, because the naive reading of "stronger wind" predicts the opposite of what happens.

2.1 SPREAD

A cell that is not burning is ignited by each burning cell within a Moore radius of 3, independently, with a weight falling off as the inverse square of the distance:

$$w(s, s') = d(s, s')^{-2} \quad \text{for } d \leq 3, \quad 0 \text{ otherwise}$$

$$P(s \text{ ignites}) = 1 - \prod_{s' \text{ burning}} (1 - w(s, s'))$$

The wind does not add a term. It classifies each burning neighbour as lying along the wind axis or not — `on_wind()` counts only the three collinear downwind offsets — and moves its weight towards 1 or towards 0 by the fraction `MU`:

on the wind axis: $w \rightarrow w + \mu(1 - w)$

everywhere else: $w \rightarrow w(1 - \mu) \leftarrow \text{exactly a factor of } (1 - \mu)$

The second line is the important one, and it is exact: **every off-axis weight in the kernel is multiplied by $(1 - \mu)$** , reaching zero only when μ is exactly 1. Meanwhile the three on-axis weights are pushed to certainty. Written out for the neighbours that matter:

MU	0	0.25	0.5	0.75	0.85	0.9	0.95	0.975	0.99	1.0
downwind d = 1	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
downwind d = 3	0.111	0.333	0.556	0.778	0.867	0.911	0.956	0.978	0.991	1.000
crosswind d = 1	1.000	0.750	0.500	0.250	0.150	0.100	0.050	0.025	0.010	0.000
front speed, cells/update	1.44	1.96	2.39	2.74	2.85	2.90	2.95	2.98	2.99	3.00

Read off `fire_spread.cell_weight()` itself rather than worked by hand. The last row is the expected distance the downwind front advances per fire update from a single burning cell, $3w_3 + 2w_2(1 - w_3) + (1 - w_2)(1 - w_3)$.

The wind makes the fire faster and thinner at the same time. It roughly doubles the downwind front speed — from 1.44 cells per update to a guaranteed 3.00 — while scaling everything off the axis by $(1 - \mu)$. Anyone expecting a stronger wind to be more dangerous is reading the first half of that sentence. The measurements in §5 say the second half wins.

2.2 WHY THINNESS BEATS SPEED: THE FIRE IS A WAVE

A burning cell in this model burns for *one fire update* and then stops. It loses `BURNING_RATE` fuel, ignites what it can reach, and goes out. The fire is therefore a wave a few cells deep travelling across the grid, not a growing region of flame with a burning interior.

That matters because of how the home base is destroyed. The base is not destroyed by fire reaching it; it accumulates one point of damage per step on which any of its four cells is alight, and is lost at `BHP` = 4 of them. What kills the base is therefore *dwell* — how many consecutive steps the wave is standing on the footprint — and a wide slow fire dwells while a narrow fast one passes over and is gone.

THE MECHANISM, IN ONE SENTENCE

Wind does not keep the fire away from the base. It makes the fire arrive sooner, cross the base in two or three steps instead of four or more, and leave. The base survives because it was singed rather than besieged.

This is visible at the extreme. At $\mu = 1$ the fire is a single line of cells one wide, advancing three cells an update along one axis, and §7.3 shows that a line lit directly beside the base can clear the two-deep footprint in its first leap — scoring two or three damage against a requirement of four, and losing a run it geometrically should have won.

3. Parameters

Every arm holds the twelve settings of the no-UAV baseline fixed and changes nothing but the wind. The baseline is therefore not a separate scenario to be compared against loosely; it is the $\mu = 0$ face of this same surface, and §6 checks that it behaves like one.

SETTING	BASELINE	HERE
ACTIVATE_WIND	False	True
FIXED_WIND	—	True (cardinal) / False (diagonal)
WIND_DIRECTION	—	north, south, east, west
FIRST_DIR, SECOND_DIR	—	the four perpendicular pairs
FIRST_DIR_PROB	—	0.5 — the even split that puts the resultant on the diagonal
MU	—	0, 0.25, 0.5, 0.75, 0.85, 0.9, 0.95, 0.975, 0.99, 1.0
WIDTH, HEIGHT	100, 100	unchanged
BATCH_SIZE	100	unchanged
NUM_AGENTS	0	unchanged
ACTIVATE_FIREFIGHTING, BHP	True, 4	unchanged
FIRE_START_POSITION, _STEP	random, 0	unchanged
BURNING_RATE, FIRE_SPREAD_SPEED	1, 1	unchanged
DENSITY_PROB	1.0	unchanged
ACTIVATE_SMOKE	False	unchanged

Everything else took its `config.py` default. `config-used.json` records all 90 settings as they were actually applied, together with the commit, so a later reader can diff rather than assume. Note that smoke is off throughout: `SMOKE_MU` is a second, independent wind, and leaving it on would have made this a two-factor experiment reported as one.

4. Method

4.1 THE SWEEPS

Each sweep is a cartesian product of its axes run through `tools/sweep.py scan`, which puts every arm in its own `headless.py` subprocess. The subprocess matters: applying overrides mutates the global `config` module, so arms sharing an interpreter leak settings into one another unless every arm happens to set every key any other arm sets.

SWEEP	VARIABLES	ARMS	RUNS/ARM	SEED BLOCK	STATUS
cardinal	<code>WIND_DIRECTION</code> × <code>MU</code>	40	1 000	3 000 000	complete
diagonal	<code>FIRST_DIR</code> × <code>SECOND_DIR</code> × <code>MU</code>	33 of 40	1 000	3 000 000	3 of 4 pairs
runlength	<code>BATCH_SIZE</code> × <code>MU</code>	32	500	4 000 000	not run (§8)
steadiness	<code>FIRST_DIR_PROB</code> × <code>MU</code>	12	500	5 000 000	not run (§8)
extent	<code>MU</code> , no home base	10	200	6 000 000	not run (§8)

The blocks are disjoint from each other and from every other experiment in the repository: the calibration sweep used 1 000 and 500 000, and the no-UAV baseline used 1 000 000 and 2 000 000.

4.2 INDEPENDENCE, AND PAIRING

Run i of an arm is seeded `base_seed + i`, and `seed_simulation()` replaces `config.SYSTEM_RANDOM` — the only generator the simulation draws from — with a fresh `random.Random(seed)` before each model is built. No state crosses between runs, and `analyse.py` refuses an arm that repeats a seed.

Because the two direction sweeps share one block, their arms are also *paired*: run 37 is the same fire in all 73 of them, so a difference between two arms is a difference between the winds and not between the fires they were shown. §7.1 verifies that rather than assuming it, and explains why it survives the extra random draw a composed wind takes.

4.3 WHY THE MU LEVELS ARE NOT EVENLY SPACED

A pilot of 100 runs per arm found the loss rate at 100% from $\mu = 0$ to $\mu = 0.75$ and at 2% at $\mu = 1.0$. An evenly spaced grid would have spent nine tenths of its arms re-measuring the same flat 100%, so the ten levels put four below 0.85 — enough to establish that the flat region is genuinely flat, in every direction — and six between 0.85 and 1.0, where the response moves. Every chart in §5 therefore plots `MU` on an evenly spaced categorical axis; a linear one would compress the entire interesting region into the last twentieth of the width.

4.4 INTERVAL

Every rate is reported with a 95% Wilson score interval. Many of these arms sit at or very near a boundary — 100%, or a fraction of a percent — which is exactly where the normal approximation misbehaves: at $\hat{p} = 1$ its standard error is zero and it collapses to the degenerate point [100%, 100%]. `wilson()` is imported from `tools/sweep.py` rather than reimplemented, so this experiment, the no-UAV baseline and the repository's sweep tool cannot disagree about what a 95% interval is.

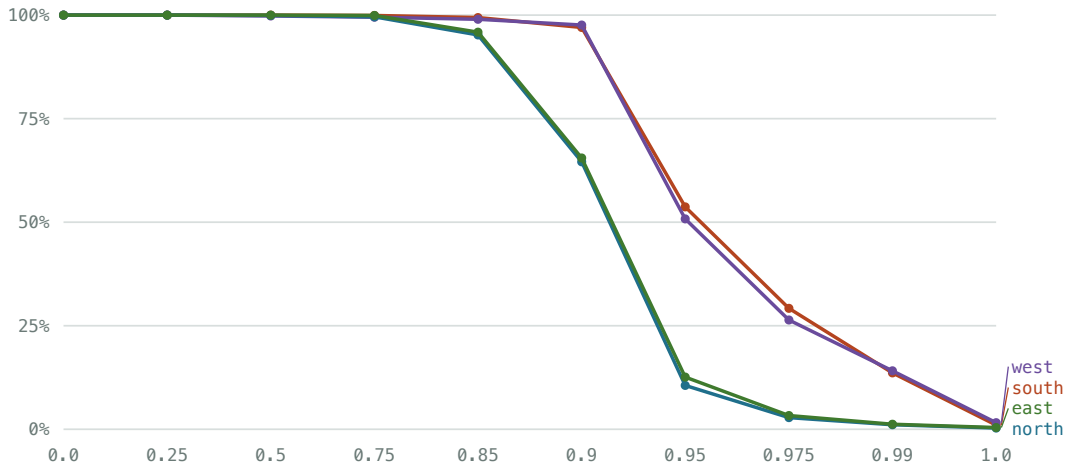
4.5 READING THE ARMS OFF DISK

`analyse.py` reads the per-arm JSON that `headless.py` writes, not the merged `runs.csv` that `tools/sweep.py` writes when a scan finishes. The distinction matters here: this sweep was interrupted, and an interrupted scan leaves every arm it completed on disk with no CSV at all. Those arms are perfectly good measurements and the identity of each is in its filename. An arm containing any errored run is discarded whole (§7.4).

5. Results

5.1 THE FOUR CARDINAL DIRECTIONS

40 arms, 1 000 runs each, all 40 000 runs drawn from one seed block so that run i is the same fire in every arm.



LOSS RATE AGAINST WIND STRENGTH. μ RUNS ON AN EVENLY SPACED CATEGORICAL AXIS, NOT A LINEAR ONE — THE LEVELS THEMSELVES ARE UNEVENLY SPACED, FOUR BELOW 0.85 AND SIX ABOVE IT, BECAUSE THAT IS WHERE THE RESPONSE LIVES.

The surface has three regions, and only the third is the one anybody would have predicted.

Below $\mu = 0.75$ the wind does not exist, as far as the base is concerned. Every one of the sixteen arms at $\mu \in \{0, 0.25, 0.5, 0.75\}$ lost at least 99.5% of its runs, against a no-wind baseline of 100.0%. By $\mu = 0.75$ the wind has thrown away three quarters of every off-axis spread weight and raised the front speed from 1.44 to 2.74 cells per update — it has transformed the fire — and the loss rate has moved by a tenth of a point.

Between 0.85 and 0.99 the direction decides everything. The four curves split into two pairs that are nowhere near each other: at $\mu = 0.9$ a south or west wind still loses 97% of runs while a north or east wind loses 65%, and at $\mu = 0.95$ it is 51–54% against 11–13%. A factor of five, from nothing but which way the air is moving, on identical fires.

At $\mu = 1$ everything collapses together, to between 0.3% and 1.6%. The fire is a single line one cell wide, and §7 accounts for every run that is lost, individually.

MU	SOUTH		WEST		NORTH		EAST	
	LOSS	95% WILSON	LOSS	95% WILSON	LOSS	95% WILSON	LOSS	95% WILSON
0.0	100.0%	99.6–100.0	100.0%	99.6–100.0	100.0%	99.6–100.0	100.0%	99.6–100.0
0.25	100.0%	99.6–100.0	100.0%	99.6–100.0	100.0%	99.6–100.0	100.0%	99.6–100.0
0.5	100.0%	99.6–100.0	99.8%	99.3–99.9	99.9%	99.4–100.0	100.0%	99.6–100.0
0.75	99.9%	99.4–100.0	99.5%	98.8–99.8	99.6%	99.0–99.8	99.9%	99.4–100.0
0.85	99.4%	98.7–99.7	99.0%	98.2–99.5	95.2%	93.7–96.4	95.9%	94.5–97.0
0.9	97.0%	95.7–97.9	97.6%	96.5–98.4	64.6%	61.6–67.5	65.5%	62.5–68.4
0.95	53.7%	50.6–56.8	50.8%	47.7–53.9	10.6%	8.8–12.7	12.6%	10.7–14.8
0.975	29.2%	26.5–32.1	26.4%	23.8–29.2	2.8%	1.9–4.0	3.3%	2.4–4.6
0.99	13.6%	11.6–15.9	14.1%	12.1–16.4	1.1%	0.6–2.0	1.2%	0.7–2.1
1.0	1.0%	0.5–1.8	1.6%	1.0–2.6	0.3%	0.1–0.9	0.4%	0.2–1.0

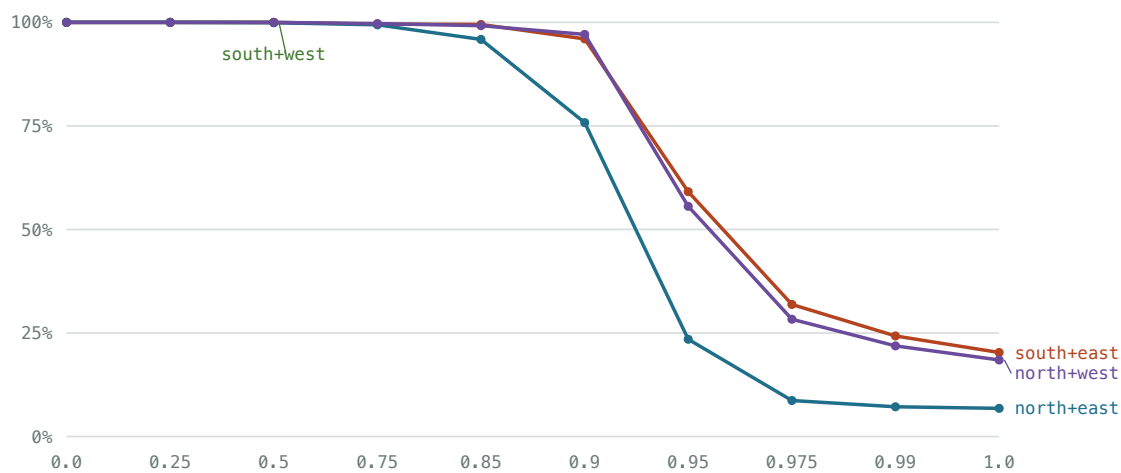
LOSS RATE AND 95% WILSON INTERVAL, 1 000 RUNS PER CELL. SHADED ROWS ARE THOSE WHERE AT LEAST ONE DIRECTION HAS MOVED OFF 100%.

The split between the directions is the map, not the wind. The home base sits at (25, 25) on a 100×100 grid — a quarter of the way in, not in the middle. A wind that drives fire towards decreasing coordinates has 73 rows of grid upwind of the base to draw ignitions from; one driving the other way has 25. South and west are the long-approach directions, north and east the short ones, and that 73:25 ratio is the whole of the effect. It would vanish on a map with a centred base, which is worth knowing before this result is carried anywhere else.

It is worth being precise about what this does and does not say. It is not that a north wind blows the fire away from the base: at $\mu = 0.9$ the off-axis weight is still 0.1, the fire is a broad wedge rather than a line, and fires lit well off the axis still arrive. It is that the wedge opens downwind, so the set of ignition cells that can feed it is three times larger for a south wind than a north one. The wind sets the shape; where the base sits on the map sets how much of the map that shape can catch.

5.2 THE FOUR DIAGONALS

There is no diagonal wind direction in this simulator. Composing two perpendicular ones is how `config.py` says to ask for one, and it is a *mixture* rather than a bearing: each cell/neighbour pair blows `FIRST_DIR` with probability `FIRST_DIR_PROB` and `SECOND_DIR` otherwise. That distinction turns out to matter more than the bearing does.



LOSS RATE AGAINST WIND STRENGTH FOR THE COMPOSED WINDS, `FIRST_DIR_PROB` = 0.5. THE `SOUTH+WEST` PAIR IS MISSING ABOVE `MU` = 0.5: THE SWEEP WAS STOPPED BEFORE THOSE ARMS RAN (§8).

A diagonal wind is far less protective than a cardinal one. At full strength a cardinal wind leaves a loss rate of 0.3 – 1.6%; a diagonal one leaves 6.8 – 20.3%, up to twenty times higher. A cardinal wind at $\mu = 1$ collapses the fire to a line one cell wide. A composed one cannot: even at $\mu = 1$ each pair of cells independently picks one of two directions, so the fire still steps two ways and stays two-dimensional.

MU	SOUTH+EAST		SOUTH+WEST		NORTH+EAST		NORTH+WEST	
	LOSS	95% WILSON	LOSS	95% WILSON	LOSS	95% WILSON	LOSS	95% WILSON
0.0	100.0%	99.6–100.0	100.0%	99.6–100.0	100.0%	99.6–100.0	100.0%	99.6–100.0
0.25	100.0%	99.6–100.0	100.0%	99.6–100.0	100.0%	99.6–100.0	100.0%	99.6–100.0
0.5	100.0%	99.6–100.0	100.0%	99.6–100.0	99.9%	99.4–100.0	100.0%	99.6–100.0
0.75	99.6%	99.0–99.8	—	—	99.4%	98.7–99.7	99.7%	99.1–99.9
0.85	99.5%	98.8–99.8	—	—	95.9%	94.5–97.0	99.2%	98.4–99.6
0.9	96.0%	94.6–97.0	—	—	75.8%	73.0–78.4	97.1%	95.9–98.0
0.95	59.1%	56.0–62.1	—	—	23.5%	21.0–26.2	55.6%	52.5–58.7
0.975	31.9%	29.1–34.9	—	—	8.7%	7.1–10.6	28.3%	25.6–31.2
0.99	24.3%	21.7–27.1	—	—	7.2%	5.8–9.0	21.9%	19.4–24.6
1.0	20.3%	17.9–22.9	—	—	6.8%	5.4–8.5	18.5%	16.2–21.0

LOSS RATE AND 95% WILSON INTERVAL, 1 000 RUNS PER CELL.

5.3 WHAT THE WIND DOES TO THE FIRE, AS OPPOSED TO THE BASE

A loss rate that does not move invites the reading that the setting is not connected. It is connected; it is doing a great deal to the fire, and none of it to the outcome. Mean cells burned per run under a south wind, against the loss rate of the same arms:

MU	0	0.25	0.5	0.75	0.85	0.9	0.95	0.975	0.99	1.0
cells burned	5 895	5 683	5 365	4 727	4 111	3 560	1 944	1 057	520	37
loss rate	100.0	100.0	100.0	99.9	99.4	97.0	53.7	29.2	13.6	1.0

Between $\mu = 0$ and 0.75 the wind removes a fifth of the burned area and changes the loss rate by one tenth of one point. By $\mu = 1$ the fire burns 37 cells of a 10 000 cell grid — a line — and still destroys the base in one run in a hundred.

A CAVEAT ON THIS TABLE

A run that is lost stops early, so burned area here mixes the extent of the fire with how long the run lasted. The comparison is sound across the top of the table, where the median step of loss moves only from 28 to 30, and progressively less so below it. The clean version of this measurement is the extent sweep described in §8, which runs with no base at all so that every run lasts the full hundred steps. It has no data yet.

6. Comparison with the baseline

The baseline lost the home base in 5 000 runs out of 5 000, a rate of 100.00% with a 95% Wilson interval of 99.92 – 100.00. Everything in §5 is a departure from that number, and this section is about how far the departures go and how much they can be trusted.

6.1 THE TWO EXPERIMENTS CANNOT BE PAIRED

The baseline drew its fires from seeds 1 000 000 upwards; this experiment draws from 3 000 000. That is deliberate. Sharing a seed block *across* experiments is not pairing, it is fitting every measurement ever taken to one set of fires, and the repository's convention is that each batch gets its own block. The consequence is that the comparison here is between independent samples, and a difference of a few tenths of a point between the baseline and a $\mu = 0$ arm is a difference between two sets of fires rather than a finding.

The honest internal comparator is therefore the $\mu = 0$ arms: the same code, with the wind switched on and turned all the way down, on this experiment's own fires. If those do not reproduce the baseline, nothing else here can be read against it.

ARM	N	LOST	LOSS RATE	95% WILSON
baseline, no wind at all	5 000	5 000	100.00%	99.92 – 100.00
wind on, $\mu = 0$, all four directions	1 000	1 000	100.00%	99.62 – 100.00
$\mu = 0.25$, worst of four	1 000	1 000	100.00%	99.62 – 100.00
$\mu = 0.5$, worst of four	1 000	998	99.80%	99.27 – 99.95
$\mu = 0.75$, worst of four	1 000	995	99.50%	98.83 – 99.79
$\mu = 0.9$, south	1 000	970	97.00%	95.75 – 97.89
$\mu = 0.9$, north	1 000	646	64.60%	61.59 – 67.50
$\mu = 1.0$, north	1 000	3	0.30%	0.10 – 0.88

6.2 HOW FAR UP THE RANGE THE BASELINE SURVIVES

Ten of the forty cardinal arms have intervals that contain the baseline's 100.0% outright: all four directions at $\mu \leq 0.25$, and three of four at $\mu = 0.5$. Reading a little more loosely, every arm up to $\mu = 0.75$ sits within half a percentage point of it.

For the purposes of comparing UAV policies, switching the wind on below $\mu = 0.75$ buys nothing. The scenario is exactly as unwinnable as the baseline, and just as saturated: it still cannot distinguish a policy that saves 0 runs in 1 000 from one that saves 1. If a later experiment needs headroom at the weak end, the wind has to be turned to 0.9 or above — and at that point the direction becomes a first-order experimental factor that has to be held fixed and reported, not left at whatever `config.py` happens to say.

6.3 THE CALIBRATION EXPERIMENT'S ASIDE, RESOLVED

The earlier calibration batch recorded that stronger wind made the scenario easier, and attributed it to the wind concentrating the fire into a ray that misses a 2×2 base. The direction of that finding is confirmed here and the explanation is very nearly right, but it is worth sharpening on two points, because both change what it implies.

- **It is not about missing.** At $\mu = 1$ the fire does reach the base in every run where the geometry allows it — 32 of 32, §7.3. What changes is that the fire that arrives is a line, and a line does not dwell long enough to do four points of damage. Missing and passing through look the same in the loss rate and are different in every other respect, not least what a UAV would have to do about them.
- **The effect is not gradual.** "Stronger wind is easier" suggests a dial. Three quarters of the dial does nothing whatsoever, and the entire effect is packed into the last fifth. Anyone tuning difficulty with **MU** would find it inert and then, in one step, decisive.

7. Validity

Most of this surface is flat, and a flat surface is exactly what a wind setting that silently failed to apply would also produce. Four checks separate those two readings. All four pass.

7.1 THE ARMS SEE THE SAME FIRES

Every arm of both direction sweeps shares one seed block, so run i should be the same fire throughout, and a difference between two arms should be a difference between the winds rather than between the fires they were shown. This holds for a reason particular to this simulator: `WildFireModel.reset()` draws the ignition cell at `model.py:88` and only builds the `FireSpread` at `model.py:109`, so the one draw a composed wind takes from `SYSTEM_RANDOM` to seed its own generator lands after the cell has been chosen. The wind changes how a fire spreads without changing where it starts.

Verified rather than assumed: 1 000 seeds, each naming one ignition cell, across all 73 arms of the two sweeps.

7.2 AT `MU` = 0 THE DIRECTION CANNOT MATTER

Zero strength multiplies every weight in the kernel by one, so the four directions at $\mu = 0$ must agree *run for run* — not merely in rate. Anything less would mean a direction is reaching the kernel by some path `MU` does not gate, and every difference reported at higher strength would be suspect.

Verified: the four arms are identical run for run, 1 000 runs each, and lose 100.00% of them (95% Wilson 99.62 – 100.00). Four identical arms is the price of being able to say the knob is wired up.

7.3 AT `MU` = 1 THE OUTCOME IS PREDICTABLE RUN BY RUN

At full strength the three on-axis weights are 1 and every other weight is 0, so the fire is a straight line advancing three cells an update. It should therefore be lost *if and only if* the ignition cell lies on the two-cell strip running through the base, on the upwind side of it. That is a prediction about which individual runs are lost, not about how many, and it is much harder to satisfy by accident than a matching rate.

DIRECTION	UPWIND STRIP	PREDICTED LOST	ACTUALLY LOST	MISMATCHES
north	50 cells	3	3	0
east	50 cells	4	4	0
south	146 cells	10	10	0
west	146 cells	15	15	0

32 predicted, 32 lost, not one mismatch in either direction. The 50-against-146 split in the second column is the 25-against-73 approach asymmetry of §5.1, and it is why north and east end lower than south and west at $\mu = 1$.

The rule is stated for ignitions further than three cells from the base, and that exclusion is not bookkeeping — it is the dwell mechanism showing itself. The wave ignites everything within three cells along the axis in its first update, so a fire lit inside that radius clears the two-deep footprint in a single leap and scores two or three damage against the four required. Three runs fell inside it and one was lost. A fire lit beside the base is *less* dangerous than one lit twenty cells away, which is absurd as physics and exactly right as a description of this model.

7.4 AN ARM WAS DISCARDED

The `south+west` arm at $\mu = 0.75$ has 1 000 records on disk of which 256 carry a `BrokenProcessPool` error, because the sweep was killed while that arm was running. `analyse.py` discards the whole arm rather than the failed runs: keeping the survivors would report a rate over whichever runs happened to finish first, which is a biased sample dressed as a measurement. A run that raised is not a measurement, and counting it either way is how an experiment reports a bug as a result.

8. What was not run

The sweep was stopped part way. Three further sweeps were designed, are still in `run.sh`, and have no data. They are listed here rather than quietly dropped, because two of them bear directly on how far the results above can be carried.

SWEEP	VARIES	WHAT IT WOULD SETTLE
runlength	<code>BATCH_SIZE</code> × <code>MU</code> , 32 arms	Whether "wind does nothing below 0.85" is a fact about wind or about a 100 step budget. The wind narrows the fire but also speeds it up, and 100 steps is more time than either version needs; a shorter run should move the cliff. The one worth running next.
steadiness	<code>FIRST_DIR_PROB</code> × <code>MU</code> , 12 arms	How much of the diagonal result in §5.2 is the mixture rather than the bearing. At <code>FIRST_DIR_PROB</code> = 1 a composed wind is a fixed wind, which would also check the two wind code paths against each other.
extent	<code>MU</code> , no base, 10 arms	The unfounded version of §5.3: with no base to lose, every run lasts the full hundred steps and burned area measures the fire rather than the run.

Within the sweeps that did run, the `south+west` diagonal is missing at $\mu \geq 0.75$ — three arms never started and one was discarded (§7.4). The three complete diagonal pairs are enough to support the finding in §5.2, which is a large effect, but the fourth pair would have made it symmetric with the cardinal sweep.

None of this affects the cardinal sweep, which is complete: 40 arms, 1 000 runs each, all four directions at all ten strengths.

9. Reproducing

Everything below runs from the repository root at the commit on the title page. The sweeps are one script, because the seed blocks and the base settings have to agree between them and a list of commands in a README is a poor way to guarantee that:

```
# all five sweeps. The two reported here are the first two, and take about
# an hour on eight workers; the other three were not run.
bash experiments/20260810_wind/run.sh

# arms.csv, summary.json, and the four checks of section 7
python3 experiments/20260810_wind/analyse.py

# config-used.json: all 90 settings as applied, the axes, and the commit
python3 experiments/20260810_wind/dump_config.py

# the charts and tables in section 5, regenerated from summary.json
python3 experiments/20260810_wind/figures.py all
```

Two things about the run are easy to get wrong and expensive to notice late. **Set the run length through `BATCH_SIZE`, never `--steps`** — they are aliases for one setting, and a sweep that varies both varies neither predictably. **`--policy random` is recorded for completeness only:** with `NUM_AGENTS = 0` no policy is ever consulted, and naming one keeps the command identical to the baseline's.

The figures in §5 are generated rather than typed. 73 arms is more numbers than can be transcribed into HTML by hand without introducing an error that no amount of proofreading finds, so `figures.py` emits the tables and the SVG charts from `summary.json` and they are pasted in; regenerating them after a re-run is a diff rather than a re-read.

The raw per-run JSON and the merged CSVs are not in the repository. `.gitignore` excludes `experiments/**/*.*json` and `experiments/**/*.*csv` on the grounds that they run to hundreds of megabytes and are regenerable from the script above, so this report and the README beside it are the record. Every number quoted in either of them is in one of the two.